

TubeBench: A Benchmark for Long-Form Media Interaction in Browser-Use Agents

Venkata Hemanth Athota

July 2026

Abstract

Browser-use agents and AI co-work systems are increasingly asked to operate inside long-form media workflows: finding moments in recordings, controlling players, using transcripts and captions, preserving browser state, and justifying answers from observed evidence. A simple request such as “pause the active player without disturbing anything else” already requires the agent to identify the right tab, mutate only the permitted media state, verify the result, and leave an auditable trace. TubeBench introduces a harness-agnostic benchmark protocol for this class of browser-based media interaction. The current paper reports benchmark design evidence, deterministic diagnostic reference conditions, static protocol-validation campaigns, an exploratory 24-slot dated live-public pilot, and an independent 72-attempt repeated dated live-public campaign. The campaign completed 63 of 72 retained attempts, with 9 reviewed partial timestamp-localization outcomes and no replacements. These results support methodology and bounded dated claims, not a deterministic leaderboard, cross-agent comparison, or general competence claim. The positive result is that trace-level scoring preserves hidden failure modes that final-answer scoring would collapse: partial progress, evidence grounding, media navigation, transcript use, tool recovery, verification quality, side effects, final-answer correctness, and trajectory validity.

1 Introduction

Consider a user with several media tabs open: one lecture is paused, another player is active, and a third recording must not be touched. The user asks an agent to pause only the active player and preserve every other player state. A final message saying “done” is not enough. The agent may have paused the wrong player, lost the task tab, skipped verification, or performed the right action without producing a trace that another evaluator can audit.

The static TCE-002 validation task—short for *TubeControl Executable task 002*—compresses this failure story into a small protocol instance: pause the one playing player while preserving the other players. The same structure appears in longer media work, where a user asks an agent to find a claim in a lecture, answer from transcript evidence, compare two recordings, seek to a timestamp, or restore playback state after inspection. These tasks are not reducible to video question answering. They require browser control, temporal evidence, media mutation, protected state, and verification.

TubeBench addresses this setting as an evaluation problem for browser-use agents, AI co-work systems, agentic harnesses, interactive browser agents, and model-tool environments. The central claim is methodological: in long-form browser-media work, the path matters. A final answer alone cannot show whether the agent used the right evidence, changed the wrong player, recovered from a coordinate error, disturbed unrelated state, or lost the task surface before interaction. TubeBench is designed to preserve those hidden failure modes as scoreable evidence.

This paper makes four contributions:

1. a benchmark framing for long-form browser-media workflows that sit between static document QA, simple web QA, coding-only benchmarks, generic GUI tasks, and short-horizon browser interactions;
2. a reusable task and trace protocol covering task objective, media context, browser state, allowed tools, expected evidence, scoring rubric, trajectory capture, and failure labels;
3. trace-level scoring dimensions that distinguish outcome quality from evidence quality, browser/media state control, recovery, verification, side effects, and trace validity; and
4. bounded diagnostic, protocol-validation, exploratory pilot, and independent repeated live-public evidence showing that the evaluation substrate preserves measurement failures instead of compressing them into a single success rate.

2 Related Work and Evaluation Gap

TubeBench builds on web and GUI-agent evaluation while targeting a specific interaction regime. WebArena evaluates long-horizon web tasks in realistic sites [6]; VisualWebArena emphasizes visually grounded web work [3]; Mind2Web studies generalist web agents on real websites [1]; and OSWorld evaluates agents in broader computer environments [4]. SWE-bench motivates executable, evidence-backed evaluation in repository tasks [2]. Recent work on living-screen interfaces also highlights the volatility of media-centered environments [5].

These benchmarks leave an important gap. Static document QA can test reading but not player-state manipulation. Web QA can test navigation but often omits temporal observation and media controls. Coding benchmarks test repository-grounded edits but not browser-mediated evidence gathering. Generic GUI tasks test interface control but may not require transcript use, caption availability checks, timestamp localization, state restoration, or media-specific side-effect accounting. TubeBench focuses on the intersection: agents must navigate long-form content over time, interact with media controls, ground claims in observable evidence, and leave a replayable trajectory for audit.

3 TubeBench Task Protocol

3.1 Task Families

The deterministic fixture suite currently defines twelve task categories that exercise the intended browser-media surface without depending on unstable external pages. The categories cover identifying the active media instance, pausing one or all active players, restoring original playback state, seeking to a timestamp, changing playback speed, muting and verifying, locating a segment from chapters and transcript, answering a timestamp-localized transcript question, finding a visual-only event, comparing two long-form sources, and detecting a restored side effect. These categories are design evidence for benchmark coverage.

The task families are deliberately broader than one website. A TubeBench task can be implemented in a benchmark-owned fixture, a local media-player surface, or an approved public media setting when volatility and privacy constraints are handled separately. The common structure is the interaction contract: the agent must work through browser-visible state and produce a scoreable trajectory.

3.2 Protocol Fields

Each task specification includes:

- objective: the user-facing goal or final answer target;
- media context: videos, transcripts, chapters, visual events, metadata, and relevant time spans;
- browser state: tabs, player states, protected state, and initial conditions;
- allowed tools and actions: observation channels, permitted mutations, and forbidden mutations;
- expected evidence: transcript, caption, metadata, screenshot, player-state, DOM, or visual evidence channels;
- scoring rubric: exact predicates, ordinal or partial criteria, verification requirements, and side-effect checks;
- trajectory capture: observations, actions, browser/tool calls, state snapshots, errors, recovery attempts, and final claims; and
- failure labels: primary and contributing categories for task, grounding, verification, trace, and runtime failures.

3.3 Worked Example: TCE-002

TCE-002 expands to *TubeControl Executable task 002*, titled “Pause only the playing video.” The initial browser-visible state contains three players: A is paused and protected, B is playing and is the only permitted mutation target, and C is paused and protected. The agent must observe the player states, pause B, and verify that A, B, and C are all paused while A and C remain otherwise unchanged. Accepted evidence includes player-state, DOM player-state, or screenshot observations. The task requires no textual answer; success is determined from the final state and recorded verification. Changing A or C is a side-effect disturbance even when B is paused correctly.

Figure 1 makes the task contract explicit. TCE-002 is a deterministic protocol and handoff example; it is not part of the 72-attempt live-public campaign and should not be read as a broad competence test.

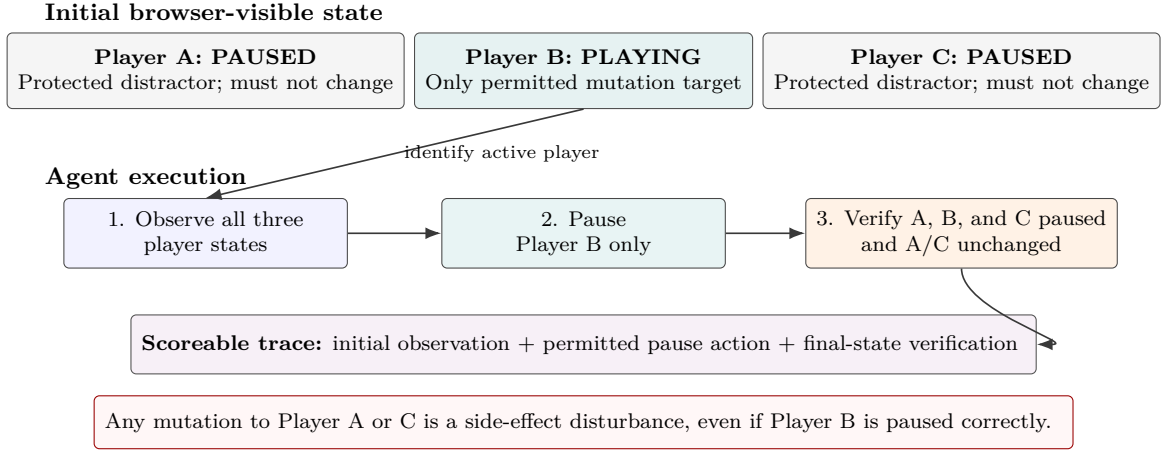


Figure 1: Worked TCE-002 example. Player B is the only mutable target; success requires all three players paused, Players A and C unchanged, and an auditable final-state verification.

3.4 Trace Evaluation and Publication Pipeline

Figure 2 separates execution evidence from paper-safe reporting. Each manifest slot produces private trace and screenshot evidence. The evaluator validates schema and pinned revisions, scores task criteria, outcome, grounding, and side effects, and retains completed, partial, failed, blocked, and invalid outcomes without replacement. Independent review approves the recorded classification, not task success. Only the reviewed aggregate crosses into generated tables, the claims ledger, and the paper.

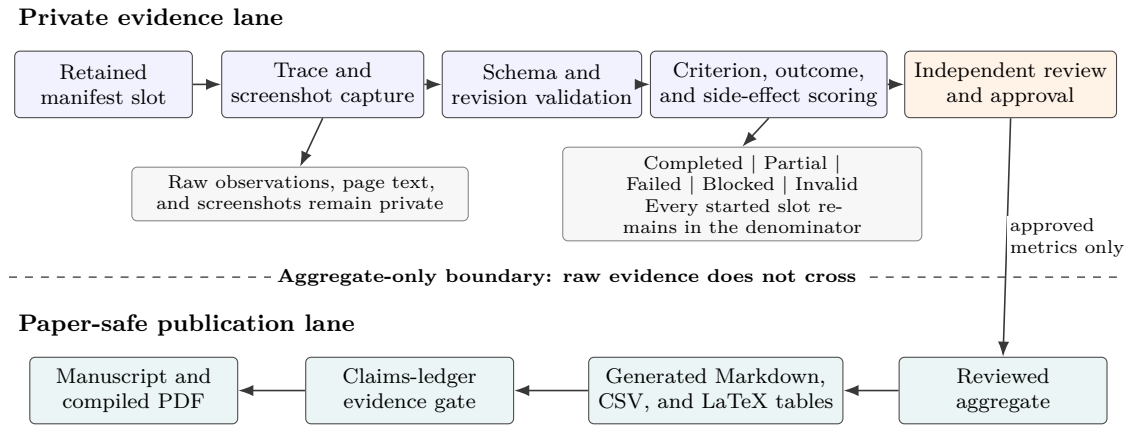


Figure 2: Trace evaluation and publication flow. Raw observations, page text, and screenshots remain private; only reviewed aggregate metrics cross the evidence boundary into generated tables, claim approval, and the compiled paper.

4 Metrics and Trace-Level Scoring

TubeBench avoids reducing browser-media work to a single final-answer score. A run can satisfy the final state while being poorly grounded, can make partial progress before a runtime blocker, or can recover from a mistaken click while still leaving a side-effect incident. Table 1 groups the reporting dimensions by the reader question they answer.

Table 1: Trace-level scoring dimensions grouped for reader interpretation.

Group	Dimension	Reader question
Outcome	Task completion; partial completion; final-answer correctness	Did the requested state or answer actually get completed, and what useful progress survived failure?
Evidence	Evidence grounding; transcript utilization; verification quality	Did the trace show the media, transcript, metadata, visual, or state evidence needed to justify the claim?
Browser and media state	Media navigation; media control; side-effect preservation	Did the agent reach the right surface, operate the right player, and avoid disturbing protected state?
Trace and audit	Tool recovery; trajectory validity; failure label	Can the run be ingested, replayed, scored, and assigned to the right model, task, harness, or runtime failure mode?

For deterministic state predicates, the evaluator records exact success:

$$S_{\text{exact}} = \mathbf{1}[\text{all required final predicates hold}].$$

The stricter disturbance-free score requires success, required evidence, and no forbidden side-effect incident:

$$S_{\text{dfs}} = \mathbf{1}[S_{\text{exact}} = 1 \wedge E_{\text{required}} = 1 \wedge I_{\text{side effects}} = 0].$$

Exact scoring is useful when the task state is fully specified. It is not sufficient for browser-media evaluation because a scoreable trajectory may include weak grounding, missing verification, partial recovery, trace-transfer failure, or a runtime blocker. Those events must remain visible in the report.

5 Experimental Evidence

5.1 Diagnostic Reference Conditions

The frozen aggregate contains four deterministic diagnostic conditions with 72 attempts each. These rows validate the scorer, aggregation code, and paper pipeline; they are diagnostic controls, not empirical performance measurements for deployed browser agents. The no-op condition executes no task mutation. The three curated references are oracle-assisted reference trajectories generated from the deterministic task catalog and scoring expectations.

Table 2: Diagnostic-only reference aggregate. These rows are pipeline controls for scorer and reporting behavior.

Condition	Attempts	Exact	DFS	Interpretation
Diagnostic-only no-op	72	4.2%	4.2%	Negative control. Its 3/72 successes come from TC-004, where doing nothing is correct state-preserving behavior.
Planner-executor reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.
Skill-augmented reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.
Hybrid reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.

Table 2 is a diagnostic-only table. Its lesson is that the publication pipeline preserves denominators and separates exact task success from disturbance-free success. This matters because a harness may complete a visible task while disturbing protected state, using insufficient evidence, or producing an invalid trace.

5.2 Static Protocol Validation

Before interpreting the live-public results, Table 3 uses two retained TCE-002 campaigns to check whether the evaluation system distinguishes readiness, task start, trace validity, and runtime blocking. These are protocol diagnostics, not agent-capability estimates. The earlier campaign retained five slots: four produced valid task-started traces and one was runtime-blocked before task interaction. The readiness-gated campaign also retained five slots: all five readiness gates passed, but all five slots were blocked before task interaction when the browser controller lost the task tab.

Table 3: Retained TCE-002 protocol diagnostics. Readiness passed in the second campaign, but task interaction did not start.

Campaign	Slots	Ready	Started	Valid	Blocked	Reader-facing interpretation
Earlier retained run	5	N/A	4	4	1	The task could start and traces could be scored, but handoff stability was not strict.
Readiness-gated run	5	5	0	0	5	Readiness passed; the failure moved to controller/tab lifecycle before task behavior.

For a retained campaign of N slots, TubeBench reports:

$$r_{\text{ready}} = \frac{n_{\text{ready}}}{N}, \quad r_{\text{valid}} = \frac{n_{\text{valid traces}}}{N}, \quad r_{\text{blocked}} = \frac{n_{\text{runtime blocked}}}{N}.$$

For the readiness-gated campaign, $r_{\text{ready}} = 1.0$, $r_{\text{valid}} = 0.0$, and $r_{\text{blocked}} = 1.0$. This is useful because it localizes failure before task interaction. Replacing blocked slots would erase the reliability signal that a browser-media benchmark must preserve.

5.3 Exploratory Live Public Video Pilot

The formal live-public result in this report is a dated one-seed pilot, not a repeated benchmark score. It uses the `live_public_video_v0` catalog: 24 retained public-page tasks across YouTube, MIT OpenCourseWare, C-SPAN, Internet Archive, and Library of Congress public-domain film pages. The run was performed on June 30, 2026 with a fresh unauthenticated browser context per retained slot. Raw traces, screenshots, page text, and observations remain in the private lab repository; the paper consumes only the reviewed aggregate export.

Table 4: Formal dated live-public pilot result. Blocked slots remain in the denominator.

Run	Attempts	Completed	Blocked	Completion	Criterion	Unsupported	Interpretation
Live public video pilot v0	24	22	2	91.7%	0.958	0	One-seed volatile public-page pilot, not a repeated benchmark score.

This result is useful because blocked public-site states stay in the denominator instead of being retried away. The pilot completed 22 of 24 retained slots, with 2 availability/volatility blocks and no unsupported claims recorded in the aggregate.

Table 5: Live-public pilot by site.

Site	Attempts	Completed	Blocked	Completion	Reader-facing signal
C-SPAN	5	4	1	80.0%	One availability/volatility block on a dynamic public-affairs page.
Internet Archive	4	4	0	100.0%	All retained public-domain moving-image pages completed.
Library of Congress	2	2	0	100.0%	Both public-domain film pages completed.
MIT OCW	5	5	0	100.0%	Lecture metadata/transcript tasks completed.
YouTube	8	7	1	87.5%	One availability/volatility block on a live-stream style task.

Table 6: Live-public pilot by task family.

Task family	Attempts	Completed	Blocked	Completion	Interpretation
Blocked/volatile reporting	3	2	1	66.7%	Volatility task retained one block as intended.
Cross-video comparison	4	4	0	100.0%	Comparison tasks completed with evidence.
Metadata inspection	6	6	0	100.0%	Public metadata inspection completed.
Timestamp localization	3	2	1	66.7%	One timestamp task exposed public-site availability volatility.
Transcript/caption availability	5	5	0	100.0%	Transcript/caption checks completed.
Visual evidence	3	3	0	100.0%	Screenshot-backed visual evidence tasks completed.

Failure taxonomy reporting remains separate from completion reporting: the aggregate records 22 traces with no failure label and 2 `availability_blocked` failures at the availability stage. This pilot is retained unchanged as exploratory evidence and is not pooled with the independent campaign below.

5.4 Independent 72-Attempt Retained Campaign

The `live-public-video-retained-v1` campaign is an independent repeated dated study performed on July 14, 2026. It instantiates the same 24-task catalog under three fresh retained repetition identifiers (17, 29, and 43), for 72 new attempts. Each attempt ran once in a fresh signed-out isolated browser context with read-only interaction. Started attempts were never reloaded, retried, or replaced; blocked and invalid outcomes would have remained in the denominator. Every retained attempt was independently reviewed against its private trace and screenshot evidence before aggregate-only export. The original pilot attempts are excluded from the campaign denominator.

Table 7: Independent retained live-public campaign overall result.

Metric	Value
Retained attempts	72
Outcome profile	63 completed; 9 partial
Completion	87.5%
Criteria passed	207 of 216 (95.8%)
Evidence coverage	100.0%
Screenshot coverage	100.0%

The campaign completed 63 of 72 retained attempts (87.5%), with 9 partial outcomes and no failed, blocked, or invalid attempts. Mean criterion score was 0.958; evidence coverage and screenshot coverage were both 100%. The reviewed aggregate records zero unsupported claims and zero side-effect incidents. These are dated results for one browser/model configuration, not a deterministic leaderboard, cross-agent comparison, or general competence claim.

Table 8: Retained live-public campaign by independent repetition.

Seed	Attempts	Completed	Partial	Completion	Criterion
17	24	21	3	87.5%	0.958
29	24	21	3	87.5%	0.958
43	24	21	3	87.5%	0.958

All three repetitions produced the same aggregate outcome: 21 completed and 3 partial attempts out of 24. At task level, 21 tasks completed in all three repetitions and 3 completed in none of the three. This stability describes the retained campaign only; it does not establish invariance across prompts, agents, browser surfaces, or future page states.

Table 9: Task-level completion stability across three retained repetitions.

Completed repetitions	Tasks
0 of 3	3
3 of 3	21

Table 10: Retained live-public campaign by site.

Site	Attempts	Completed	Partial	Completion	Criterion
C-SPAN	15	12	3	80.0%	0.933
Internet Archive	12	12	0	100.0%	1.000
Library of Congress	6	6	0	100.0%	1.000
MIT OCW	15	12	3	80.0%	0.933
YouTube	24	21	3	87.5%	0.958

Table 11: Retained live-public campaign by task family.

Task family	Attempts	Completed	Partial	Completion	Criterion
Blocked / volatile reporting	9	9	0	100.0%	1.000
Cross-video comparison	12	12	0	100.0%	1.000
Metadata inspection	18	18	0	100.0%	1.000
Timestamp localization	9	0	9	0.0%	0.667
Transcript / caption availability	15	15	0	100.0%	1.000
Visual evidence	9	9	0	100.0%	1.000

The 9 partial outcomes are exactly the three timestamp-localization tasks repeated three times. Their first criterion lacked sufficient page-visible evidence under the read-only observation contract; the remaining 207 of 216 criteria passed. The failure taxonomy therefore records 9 `timestamp_localization_failure` outcomes at the evidence-collection stage, while the other five task families completed all retained attempts.

Table 12: Outcome, criterion, failure-type, and failure-stage counts.

Category	Label	Count
Outcomes	Completed	63
Outcomes	Partial	9
Criterion statuses	Fail	9
Criterion statuses	Pass	207
Failure types	None	63
Failure types	Timestamp localization failure	9
Failure stages	Evidence collection	9

6 Results and Failure Analysis

The current evidence supports five bounded findings.

Trace-level scoring preserves hidden failures. Exact completion and disturbance-free completion must be reported separately. A run that reaches the requested final state can still be weakly grounded, unverifiable, or unsafe with respect to protected player state.

Diagnostic controls validate the evaluation substrate. The frozen reference aggregate keeps the denominator visible, distinguishes no-op behavior from curated reference behavior, and catches scoring or reporting regressions before agent comparisons are introduced.

Retained blockers identify the failure layer. The static campaigns identify trace and runtime states that are neither task failures nor successes. The readiness-gated run shows a controller/runtime failure before task behavior, which is a different finding from a media-reasoning failure.

Live-public pilots expose site volatility without replacing slots. The 24-slot live public video pilot completed 22 slots and retained 2 blocked availability/volatility outcomes. Its value is not as a deterministic leader-

board, but as a dimensional check that reports site, task-family, evidence, screenshot, unsupported-claim, and failure-stage signals together.

Repeated retention separates stable completion from stable partial evidence. The independent 72-attempt campaign completed 63 attempts and retained 9 partial timestamp-localization outcomes. The same three tasks were partial in all three repetitions, while evidence and screenshot coverage remained complete. This is a reproducibility signal inside one dated configuration, not a general capability estimate.

Table 13 gives the failure labels used to preserve these distinctions.

Table 13: Failure labels for trace-level browser-media evaluation.

Failure label	Observable signal
Task misunderstanding	Wrong media item, time span, player, or state predicate.
Navigation failure	The agent cannot reach or retain the relevant browser/media context.
Media-control failure	The wrong control is invoked or the wrong player state changes.
Evidence-grounding failure	The final claim is unsupported, partially supported, or contradicted by observed evidence.
Transcript or metadata misuse	Transcript, captions, chapters, or metadata are treated as sufficient when required evidence is absent or conflicting.
Weak verification	The agent declares completion without checking the required state or evidence.
Side-effect disturbance	Unrelated tabs, players, account-visible state, or protected browser state are changed.
Brittle recovery	A mistaken action is partly repaired but leaves state, evidence, or reasoning inconsistent.
Trace-transfer failure	Browser-visible work occurs but the trace cannot be captured, ingested, or scored.
Runtime/controller failure	Browser, tab, controller, or harness lifecycle failure prevents or interrupts task interaction.
Overconfident finalization	The final answer masks partial completion, missing evidence, or infrastructure blockers.

7 Discussion

TubeBench is most useful when an evaluator needs to decide where a failure belongs: the task, the evidence source, the browser state, the agent policy, the controller, or the trace handoff. Long-form browser-media work creates failure modes that are easy to hide behind final-answer scoring: selecting the wrong player, treating a transcript affordance as transcript evidence, confusing metadata with observed media, disturbing another media instance, or reporting success after brittle recovery. Trace-level evaluation turns those events into auditable records.

This design also changes how results should be read. A single top-line score is too small for the domain. Exact predicates remain important for deterministic state tasks, but they have to be paired with grounding, verification, side-effect, recovery, and trace-validity evidence. Ordinal scoring can capture partial progress, but it needs reviewed rubrics to avoid vague success inflation. For live media pages, transcripts, captions, ads, consent flows, recommendations, layout experiments, and live edges can change at run time, so benchmark-owned fixtures and aggregate-only promotion boundaries remain part of the methodology.

8 Limitations and Threats to Validity

Construct validity. Exact state predicates may reject acceptable behavior that the task contract did not enumerate, while ordinal criteria may overstate partial progress when evidence sufficiency is underspecified. The current task families cover media control, metadata, transcript and caption availability, timestamp localization, comparison, and visual evidence, but they do not exhaust long-horizon media research, multimodal synthesis, or collaborative workflows.

External validity. The repeated campaign evaluates one model and harness configuration, one prompt and browser setup, a limited set of public sites, English-oriented content, signed-out contexts, and read-only tasks. The result therefore does not establish behavior across agents, prompts, browsers, regions, languages, accessibility modes, authenticated workflows, or consequential mutations.

Measurement validity. Every retained campaign attempt received explicit review, but the study does not yet estimate inter-rater agreement across a larger reviewer pool. Screenshot and trace evidence can also omit context that a human observer would use, particularly for temporal localization and transient player state.

Statistical validity. Twenty-four tasks repeated three times provide descriptive within-configuration stability, not an independent population sample. The paper reports retained denominators and task-level patterns rather than confidence intervals, significance tests, or cross-agent rankings; repeated outcomes on the same task contract should not be treated as independent estimates of general competence.

Reproducibility and temporal validity. Public media pages change as transcripts, captions, recommendations, consent flows, regional availability, playback state, and live edges evolve. Dated runs and no-replacement rules preserve that volatility, while the aggregate-only privacy boundary prevents public release of raw traces and screenshots. Independent readers can verify schemas, aggregates, checksums, and generated tables, but cannot fully replay every private observation from the paper artifact alone.

9 Conclusion

TubeBench frames long-form browser-media interaction as a trace-level benchmark for browser-use agents and AI co-work systems. Its task protocol captures the objective, media context, browser state, allowed tools, evidence channels, scoring rubric, trajectory, and failure labels needed to evaluate agentic media workflows. The current diagnostic, static protocol-validation, exploratory pilot, and independent repeated dated live-public evidence shows that completion, partial evidence, grounding, media control, recovery, verification, side effects, public-site volatility, and trace validity must be reported separately. The methodological contribution is an evaluation substrate that keeps the evidence needed to measure interactive browser agents rigorously while preserving bounded claims and private/public evidence separation.

References

- [1] Xiang Deng et al. Mind2web: Towards a generalist agent for the web, 2023. URL <https://arxiv.org/abs/2306.06070>.
- [2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [3] Jing Yu Koh et al. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks, 2024. URL <https://arxiv.org/abs/2401.13649>.
- [4] Tianbao Xie et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
- [5] Jiashu Yao, Heyan Huang, Daiqing Wu, Wangke Chen, Huaxi Ai, Haoyu Wen, Zeming Liu, and Yuhang Guo. Benchmarking living-screen-native gui agents on short-video platforms, 2026. URL <https://arxiv.org/abs/2606.04701>.
- [6] Shuyan Zhou et al. Webarena: A realistic web environment for building autonomous agents, 2023. URL <https://arxiv.org/abs/2307.13854>.